

```
import os
import uuid
import shutil
from flask import Flask, render_template, request, send_file, jsonify, after_this_request

from pypdf import PdfReader, PdfWriter
import pikepdf
import fitz # PyMuPDF
from pdf2docx import Converter
from ebooklib import epub

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_DIR = os.path.join(BASE_DIR, "uploads")
OUTPUT_DIR = os.path.join(BASE_DIR, "outputs")
os.makedirs(UPLOAD_DIR, exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)

ALLOWED_PDF = {"pdf"}
MAX_CONTENT_LENGTH = 50 * 1024 * 1024 # 50 MB

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = MAX_CONTENT_LENGTH

def allowed_file(filename, allowed_exts):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in allowed_exts

def save_upload(file_storage):
    ext = file_storage.filename.rsplit(".", 1)[1].lower()
    name = f"{uuid.uuid4().hex}.{ext}"
    path = os.path.join(UPLOAD_DIR, name)
    file_storage.save(path)
    return path

def cleanup_later(*paths):
    @after_this_request
    def remove(response):
        for p in paths:
            try:
                if os.path.exists(p):
                    os.remove(p)
            except Exception:
                pass
        return response

# ----- PAGE ROUTES -----

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/merge")
def merge_page():
    return render_template("merge.html")

@app.route("/split")
def split_page():
    return render_template("split.html")

@app.route("/compress")
def compress_page():
    return render_template("compress.html")

@app.route("/convert")
def convert_page():
    return render_template("convert.html")
```

app.py (continued)

```
@app.route("/watermark")
def watermark_page():
    return render_template("watermark.html")

@app.route("/ebook")
def ebook_page():
    return render_template("ebook.html")

# ----- API: MERGE -----

@app.route("/api/merge", methods=["POST"])
def api_merge():
    files = request.files.getlist("files")
    if len(files) < 2:
        return jsonify({"error": "Kam se kam 2 PDF files chahiye"}), 400

    saved_paths = []
    try:
        writer = PdfWriter()
        for f in files:
            if not allowed_file(f.filename, ALLOWED_PDF):
                return jsonify({"error": f"{f.filename} valid PDF nahi hai"}), 400
            path = save_upload(f)
            saved_paths.append(path)
            reader = PdfReader(path)
            for page in reader.pages:
                writer.add_page(page)

        out_name = f"merged_{uuid.uuid4().hex}.pdf"
        out_path = os.path.join(OUTPUT_DIR, out_name)
        with open(out_path, "wb") as f_out:
            writer.write(f_out)

        cleanup_later(*saved_paths, out_path)
        return send_file(out_path, as_attachment=True, download_name="merged.pdf")
    except Exception as e:
        for p in saved_paths:
            if os.path.exists(p):
                os.remove(p)
        return jsonify({"error": str(e)}), 500

# ----- API: SPLIT -----

@app.route("/api/split", methods=["POST"])
def api_split():
    f = request.files.get("file")
    if not f or not allowed_file(f.filename, ALLOWED_PDF):
        return jsonify({"error": "Valid PDF file chahiye"}), 400

    ranges = request.form.get("ranges", "").strip() # e.g. "1-3,5,7-9"
    in_path = save_upload(f)

    try:
        reader = PdfReader(in_path)
        total_pages = len(reader.pages)

        writer = PdfWriter()
        if ranges:
            pages_to_extract = parse_page_ranges(ranges, total_pages)
        else:
            pages_to_extract = list(range(total_pages))

        for idx in pages_to_extract:
            writer.add_page(reader.pages[idx])

        out_name = f"split_{uuid.uuid4().hex}.pdf"
        out_path = os.path.join(OUTPUT_DIR, out_name)
        with open(out_path, "wb") as f_out:
            writer.write(f_out)
```

app.py (continued)

```
        cleanup_later(in_path, out_path)
        return send_file(out_path, as_attachment=True, download_name="split.pdf")
except Exception as e:
    if os.path.exists(in_path):
        os.remove(in_path)
    return jsonify({"error": str(e)}), 500

def parse_page_ranges(ranges_str, total_pages):
    indices = []
    parts = ranges_str.split(",")
    for part in parts:
        part = part.strip()
        if not part:
            continue
        if "-" in part:
            start, end = part.split("-")
            start = int(start.strip())
            end = int(end.strip())
            for p in range(start, end + 1):
                if 1 <= p <= total_pages:
                    indices.append(p - 1)
        else:
            p = int(part)
            if 1 <= p <= total_pages:
                indices.append(p - 1)
    return indices

# ----- API: COMPRESS -----

@app.route("/api/compress", methods=["POST"])
def api_compress():
    f = request.files.get("file")
    if not f or not allowed_file(f.filename, ALLOWED_PDF):
        return jsonify({"error": "Valid PDF file chahiye"}), 400

    in_path = save_upload(f)
    out_name = f"compressed_{uuid.uuid4().hex}.pdf"
    out_path = os.path.join(OUTPUT_DIR, out_name)

    try:
        pdf = pikepdf.open(in_path)
        pdf.save(out_path, compress_streams=True, object_stream_mode=pikepdf.ObjectStreamMode.generate,
                recompress_flate=True)
        pdf.close()

        cleanup_later(in_path, out_path)
        return send_file(out_path, as_attachment=True, download_name="compressed.pdf")
    except Exception as e:
        if os.path.exists(in_path):
            os.remove(in_path)
        return jsonify({"error": str(e)}), 500

# ----- API: CONVERT (PDF -> Word / Images) -----

@app.route("/api/convert/word", methods=["POST"])
def api_convert_word():
    f = request.files.get("file")
    if not f or not allowed_file(f.filename, ALLOWED_PDF):
        return jsonify({"error": "Valid PDF file chahiye"}), 400

    in_path = save_upload(f)
    out_name = f"converted_{uuid.uuid4().hex}.docx"
    out_path = os.path.join(OUTPUT_DIR, out_name)

    try:
        cv = Converter(in_path)
        cv.convert(out_path)
        cv.close()

        cleanup_later(in_path, out_path)
```

app.py (continued)

```
        return send_file(out_path, as_attachment=True, download_name="converted.docx")
    except Exception as e:
        if os.path.exists(in_path):
            os.remove(in_path)
        return jsonify({"error": str(e)}), 500

@app.route("/api/convert/images", methods=["POST"])
def api_convert_images():
    f = request.files.get("file")
    if not f or not allowed_file(f.filename, ALLOWED_PDF):
        return jsonify({"error": "Valid PDF file chahiye"}), 400

    in_path = save_upload(f)
    work_id = uuid.uuid4().hex
    work_dir = os.path.join(OUTPUT_DIR, work_id)
    os.makedirs(work_dir, exist_ok=True)

    try:
        doc = fitz.open(in_path)
        for i, page in enumerate(doc):
            pix = page.get_pixmap(dpi=150)
            pix.save(os.path.join(work_dir, f"page_{i+1}.png"))
        doc.close()

        zip_path = os.path.join(OUTPUT_DIR, f"images_{work_id}")
        shutil.make_archive(zip_path, "zip", work_dir)
        zip_full = zip_path + ".zip"

        cleanup_later(in_path, zip_full)

    @after_this_request
    def rm_dir(response):
        shutil.rmtree(work_dir, ignore_errors=True)
        return response

    return send_file(zip_full, as_attachment=True, download_name="pages.zip")
    except Exception as e:
        if os.path.exists(in_path):
            os.remove(in_path)
        shutil.rmtree(work_dir, ignore_errors=True)
        return jsonify({"error": str(e)}), 500

# ----- API: WATERMARK -----

@app.route("/api/watermark/add", methods=["POST"])
def api_watermark_add():
    f = request.files.get("file")
    text = request.form.get("text", "WATERMARK")
    if not f or not allowed_file(f.filename, ALLOWED_PDF):
        return jsonify({"error": "Valid PDF file chahiye"}), 400

    in_path = save_upload(f)
    out_name = f"watermarked_{uuid.uuid4().hex}.pdf"
    out_path = os.path.join(OUTPUT_DIR, out_name)

    try:
        doc = fitz.open(in_path)
        for page in doc:
            rect = page.rect
            center = fitz.Point(rect.width / 2, rect.height / 2)
            morph = (center, fitz.Matrix(45))
            page.insert_text(
                fitz.Point(rect.width / 4, rect.height / 2),
                text,
                fontsize=40,
                color=(0.7, 0.7, 0.7),
                overlay=True,
                morph=morph,
            )
        doc.save(out_path)
        doc.close()
```

app.py (continued)

```
        cleanup_later(in_path, out_path)
        return send_file(out_path, as_attachment=True, download_name="watermarked.pdf")
    except Exception as e:
        if os.path.exists(in_path):
            os.remove(in_path)
        return jsonify({"error": str(e)}), 500

@app.route("/api/watermark/remove", methods=["POST"])
def api_watermark_remove():
    """
    Removes text/image watermark LAYERS that are part of the PDF's own
    content streams (e.g. a watermark you or your tool added earlier).
    This works by detecting low-opacity / repeated overlay objects across
    pages and stripping them. It does not, and is not intended to, defeat
    DRM or licensing protections on third-party copyrighted content.
    """
    f = request.files.get("file")
    if not f or not allowed_file(f.filename, ALLOWED_PDF):
        return jsonify({"error": "Valid PDF file chahiye"}), 400

    in_path = save_upload(f)
    out_name = f"clean_{uuid.uuid4().hex}.pdf"
    out_path = os.path.join(OUTPUT_DIR, out_name)

    try:
        doc = fitz.open(in_path)
        for page in doc:
            # Remove annotation-based watermarks (stamps)
            annots = list(page.annots() or [])
            for annot in annots:
                if annot.type[1] in ("Stamp", "Watermark"):
                    page.delete_annot(annot)

            # Remove low-opacity image XObjects often used as watermarks
            for img in page.get_images(full=True):
                xref = img[0]
                try:
                    smask = doc.xref_get_key(xref, "SMask")
                    if smask and smask[0] != "null":
                        page.delete_image(xref)
                except Exception:
                    pass

            # Redact text spans that repeat identically across many pages
            # (classic diagonal "watermark" text) – heuristic: large font,
            # low fill alpha, rotated.
            blocks = page.get_text("dict")["blocks"]
            for b in blocks:
                if "lines" not in b:
                    continue
                for line in b["lines"]:
                    for span in line["spans"]:
                        if span.get("size", 0) > 28 and abs(line.get("dir", (1, 0))[1]) > 0.3:
                            page.add_redact_annot(span["bbox"])
            page.apply_redactions()

        doc.save(out_path)
        doc.close()

        cleanup_later(in_path, out_path)
        return send_file(out_path, as_attachment=True, download_name="watermark_removed.pdf")
    except Exception as e:
        if os.path.exists(in_path):
            os.remove(in_path)
        return jsonify({"error": str(e)}), 500

# ----- API: EBOOK GENERATE -----

@app.route("/api/ebook/generate", methods=["POST"])
def api_ebook_generate():
    title = request.form.get("title", "My Ebook")
    author = request.form.get("author", "Unknown")
```

app.py (continued)

```
f = request.files.get("file")

if not f:
    return jsonify({"error": "Text ya PDF file chahiye"}), 400

fname = f.filename.lower()
in_path = save_upload(f) if "." in fname else None

try:
    if fname.endswith(".pdf"):
        doc = fitz.open(in_path)
        text_content = ""
        for page in doc:
            text_content += page.get_text()
        doc.close()
    elif fname.endswith(".txt"):
        with open(in_path, "r", encoding="utf-8", errors="ignore") as fh:
            text_content = fh.read()
    else:
        return jsonify({"error": "Sirf .pdf ya .txt file allowed hai"}), 400

    book = epub.EpubBook()
    book.set_identifier(uuid.uuid4().hex)
    book.set_title(title)
    book.set_language("en")
    book.add_author(author)

    paragraphs = [p.strip() for p in text_content.split("\n") if p.strip()]
    chapter_html = "<h1>{}</h1>".format(title)
    for p in paragraphs:
        chapter_html += f"<p>{p}</p>"

    chapter = epub.EpubHtml(title=title, file_name="chap_1.xhtml", lang="en")
    chapter.content = chapter_html
    book.add_item(chapter)

    book.toc = (chapter,)
    book.add_item(epub.EpubNcx())
    book.add_item(epub.EpubNav())
    book.spine = ["nav", chapter]

    out_name = f"ebook_{uuid.uuid4().hex}.epub"
    out_path = os.path.join(OUTPUT_DIR, out_name)
    epub.write_epub(out_path, book)

    cleanup_paths = [out_path] + ([in_path] if in_path else [])
    cleanup_later(*cleanup_paths)
    return send_file(out_path, as_attachment=True, download_name=f"{title}.epub")
except Exception as e:
    if in_path and os.path.exists(in_path):
        os.remove(in_path)
    return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=False)
```